

Рендеринг Compose, оставшаяся ниша XML и почему MVI ложится на Compose — учебный конспект

TL;DR

- **Рендеринг:** «пропуск рекомпозиции» (решение рантайма Compose не перевызывать композабл) и «пропущенный/дёрганный кадр» (Choreographer/RenderThread не успели к VSYNC в бюджет ~16.6 мс @60 Гц / ~8.3 мс @120 Гц) — разные вещи; первое может быть причиной второго, но jank часто вообще не про Compose (GC, I/O, верификация классов на главном потоке).
- **XML в 2026:** View/XML/RemoteViews остаётся обязательным для виджетов (Glance→RemoteViews, стабильная 1.1.1), кастомных уведомлений, поверхностей SurfaceView (камера/видео/GL/WebRTC), WebView, экранов настроек (androidx.preference 1.2.1) и Wear Tiles — Compose-нативного ответа для них пока нет.
- **MVI+Compose:** единый неизменяемый STATE — ровно та форма «один снимок на вход → весь UI на выход», которую Compose и так хочет; разобранная production-реализация прячет RxJava-движок Badoo/Bumble MVI Core за anti-corruption layer, использует самодельную неограниченную очередь новостей ради never-drop-доставки и требует контракт «must never throw», превращая баги в «неправильное состояние», а не в краш.

Как читать этот конспект

Это третий конспект серии. Он не пересказывает ранее разобранное: механику рекомпозиции/стабильности/strong-skipping, side-effect API (LaunchedEffect/DisposableEffect/derivedStateOf), перф-плейбук (deferred reads, лямбда-модификаторы, стабильные ключи LazyColumn), а также базу MVI (что такое Actor/Reducer/NewsPublisher, однонаправленный поток данных, философию state-vs-news, базовую стратегию тестирования). Всё это предполагается известным. Там, где концепция реально трудная, сначала даётся простая бытовая аналогия, затем — точная техническая формулировка.

Key Findings

1. **Два «пропуска» диагностируются разными инструментами.** Пропуск рекомпозиции — Layout Inspector и compiler metrics; пропущенный кадр — системный трейс (Perfetto), `FrameTimingMetric`, Android Vitals. Путаница между ними — источник неверных гипотез при оптимизации.
 2. **Ниша XML в 2026 сузилась, но не исчезла.** Glance — это Compose-стиль поверх RemoteViews, а не Compose-нативный рендеринг; для камеры/видео/GL/WebRTC/WebView/карт/настроек/Tiles `AndroidView`-интероп — норма, а не временный костыль.
 3. **Вес APK от Compose нейтрален-положителен на «чистых» проектах.** На смешанной стадии миграции временно растёт (Sunflower: +782 КБ), но «чисто Compose» выходит в ноль-минус, а полная миграция Tivi дала −46% APK и −17% method count.
 4. **Baseline Profiles реально снижают старт и jank** (кейс Vodafone: −20% к старту, −38.7% janky-кадров на дашборде), но для своих экранов профиль нужно генерировать самому.
 5. **MVI закрывает конструктивно ровно те четыре точки**, где MVVM с N потоками `StateFlow` напрягается: гонки без координатора, нечитабельные `combine()`, one-shot события без дома, тесты на порядок вызовов.
 6. **Anti-corruption layer + контракт «never throw» + never-drop-очередь** — три архитектурных решения разобранной реализации, каждое с явным трейд-оффом.
-

Details

Тема 1. Конвейер рендеринга и два разных «пропуска» (skipping)

От изменения состояния до пикселя

Compose — верхний слой. Он не рисует пиксели сам: он готовит описание кадра и отдаёт его классическому конвейеру рендеринга Android. Полезно держать в голове две сцепленные машины.

Машина №1 — три фазы Compose. Когда меняется наблюдаемое состояние, Compose проходит по трём фазам:

1. **Composition** — выполняются `@Composable`-функции и строится дерево узлов (LayoutNode). Это «что показывать».
2. **Layout** — измерение и размещение: каждый узел меряет детей и расставляет их по координатам. Это «где и какого размера».
3. **Drawing** — отрисовка узлов в буфер. Это «чем закрасить пиксели».

Важная деталь: не каждое изменение проходит все три фазы. Если поменялось только смещение (offset через лямбда-модификатор), можно пропустить Composition и войти сразу в Layout; если поменялся только цвет — можно дойти до Draw. Это то, ради чего в перф-плейбуке рекомендуют откладывать чтение состояния как можно ниже по фазам.

Машина №2 — классический конвейер кадра. Три фазы Compose — это CPU-работа на главном (UI) потоке. Результат передаётся дальше:

- **Choreographer** — системный дирижёр, который получает сигнал **VSYNC** от контроллера дисплея и по нему запускает очередной цикл: обработку ввода, анимации, и затем CPU-работу (в т.ч. рекомпозицию, *measure*, *layout*).
- **RenderThread** — отдельный поток, который принимает список нарисованных команд и передаёт их GPU.
- **Бюджет кадра.** При 60 Гц на один кадр отводится ~16.6 мс, при 120 Гц — ~8.3 мс. Если и CPU-, и GPU-работа не уложились до следующего VSYNC, кадр не готов вовремя.

Два «пропуска», которые легко перепутать

(a) «Пропуск рекомпозиции» (recomposition skip). Решение уровня компилятора/рантайма Compose: не перевызывать компоабл, потому что его входы не изменились (или сравнение по стабильности показало равенство). Разобрано в предыдущих конспектах — здесь только называем.

(b) «Пропущенный/дёрганный кадр» (skipped/janked frame). Системное событие: Choreographer/RenderThread не успели к дедлайну VSYNC. Как описывает разбор из ProAndroidDev, на дисплее 60 Гц «трата 17 мс вместо 16.6 мс означает, что кадр не готов вовремя», система показывает предыдущий кадр ещё один цикл, и это воспринимается как **jank** — заметный провал плавности (например, кратковременный переход 60→30 FPS).

Связь и различие. Отсутствие пропуска рекомпозиции (то есть компоабл всё-таки перевыполняется, хотя мог бы не перевыполняться) — **одна из возможных причин** пропущенного кадра: лишняя работа на UI-потоке съела бюджет. Но это **не одно и то же**, и (b) может случиться по причинам, вообще не связанным с рекомпозицией: тяжёлое `measure`/`layout`, дорогая отрисовка, сборка мусора (GC-пауза), тяжёлая **не-Compose работа** на главном потоке (синхронный I/O, парсинг JSON, инициализация в `onCreate`), верификация классов при первом запуске. В разобранном на ProAndroidDev случае навигационного jank «главный поток был полностью занят рекомпозицией и верификацией классов, оставив RenderThread ждать более 1.5 секунды» — jank виден в системном трейсе как красный кадр независимо от того, что его вызвало.

Практический вывод: «рекомпозиция не была пропущена» и «кадр пропущен» диагностируются разными инструментами. Первое — Layout Inspector (счётчики *recomposition/skip*) и Compose compiler metrics; второе — системный трейс (Perfetto), `FrameTimingMetric` в Macrobenchmark, Android Vitals.

Тема 2. XML в 2026: где до сих пор нет хорошего ответа у Compose

Это не гайд по миграции и не про механику `AndroidView`. Это честный перечень, где View/XML/RemoteViews по состоянию на середину 2026 остаются прагматичным или единственным выбором.

Виджеты домашнего экрана (Glance). Glance — это Compose-стиль поверх RemoteViews, а не Compose-нативный рендеринг. Стабильная версия — Glance 1.1.1, есть релиз-кандидат 1.2.0-rc01 и альфа 1.3.0-alpha02 в разработке. Официальная документация («Build UI with Glance») прямо говорит, что Glance «restricted by the limitations of AppWidgets and RemoteViews» и потому использует другой набор композаблов, чем Compose UI, и **не interoperable** с обычным Compose UI (мешать их нельзя). Практические следствия ограничений RemoteViews: нет произвольного кастомного рисования (canvas), ограниченный набор компонентов, LazyColumn транслируется в настоящий ListView (действуют те же ограничения коллекций RemoteViews), кастомные шрифты приложения не поддерживаются (fontFamily — только системные). Полностью Compose-нативного рендеринга виджетов (без RemoteViews) в шипнутом виде на 2026 нет: в Android 16 появились RemoteViews.DrawInstructions и движок RemoteCompose, но опубликованные артефакты Glance всё ещё эмитят RemoteViews — это forward-looking, не shipped. (Android Developers + 6)

Кастомные уведомления. Кастомная разметка уведомлений по-прежнему требует RemoteViews (DecoratedCustomViewStyle / setCustomContentView); Compose-эквивалента нет, потому что уведомление рендерит System UI в отдельном процессе. Более того, официальный гайд по Live Update-уведомлениям советует **избегать** кастомных уведомлений на RemoteViews (несогласованность поведения между версиями и вендорами). Роль Compose ограничена deep-link'ом в Compose-экран. (Desarrollolibre) (Android Developers)

Высокопроизводительные поверхности с кастомным рисованием. SurfaceView и всё, что рендерит в выделенную поверхность, композируемую SurfaceFlinger — камера (CameraX PreviewView), видео (media3/ExoPlayer), OpenGL/Vulkan, WebRTC — должны хоститься через AndroidView-интероп. Официальный референс AndroidView прямо называет WebView, SurfaceView, AdView как примеры, для которых «нет соответствующего Compose API». Даже «композабельные» обёртки (CameraXViewfinder, media3 PlayerSurface) внутри построены на AndroidExternalSurface/AndroidEmbeddedExternalSurface, которые оборачивают SurfaceView/TextureView. Причина фундаментальная: у Compose нет собственного эквивалента выделенной поверхности, композируемой SurfaceFlinger. PreviewView в режиме PERFORMANCE использует SurfaceView с выделенной поверхностью, чтобы попасть в hardware overlay.

(Composables + 2)

WebView. Требуется обёртки в AndroidView; Compose-нативного WebView в 2026 нет (официальный гайд советует не мигрировать сложные WebView-разметки, пока Compose-эквивалент недоступен). (Android Developers)

Экраны настроек (Preferences). androidx.preference остаётся View/Fragment-based (стабильная 1.2.1, XML <PreferenceScreen>, PreferenceFragmentCompat). Официальной Compose-библиотеки настроек нет; разработчики пишут вручную или берут сторонние.

(Android Developers)

(Medium)

Wear OS Tiles. Определяются декларативно через protoLayout/tiles, а не через Compose («Tiles не используют Compose и не являются composable»). ProtoLayout — «Compose-inspired», но не Compose. (Android Developers) (Android Developers)